

SESSION 04 / 10

# テクスチャの貼り付け

画像で質感とディテールを与える

Week 4    2 hours

# 本日のアジェンダ

01

## テクスチャの基本を理解する

読み込み・貼り付け・繰り返しの基礎

30 min

02

## 各種マップを使い分ける

normal・roughness などPBRマップ

30 min

03

## ハンズオン演習

立方体に木目テクスチャを貼る

50 min

解説 60 min → ハンズオン 50 min → 振り返り 10 min

# 前提知識・準備物

## 第3回の到達点

- Standard マテリアルで質感を設定できる
- DirectionalLight と影を扱える
- 3点照明で立体感を出せた
- animate() で立方体が回転している

## 今回必要なもの

### 前回のプロジェクト

my-threejs-app をそのまま使用

### テクスチャ画像

木目などの jpg / png を1枚用意

### マテリアルの基礎

Standard を理解していればOK

# テクスチャを貼る3つのステップ

## Texture

読み込み

画像ファイルを  
TextureLoader で  
読み込む

## Map

貼り付け

material.map に  
設定して表面に  
貼り付ける

## UV / Repeat

調整

座標と繰り返しで  
貼り方を整える

読み込み → 貼り付け → 調整 の流れで、表面の質感が決まる

# TextureLoader — 画像を読み込む

画像ファイルを非同期で読み込み、テクスチャとして使えるようにする

```
const loader = new THREE.TextureLoader();  
const tex = loader.load('wood.jpg');  
tex.colorSpace = THREE.SRGBColorSpace;
```

## load

画像のパスを渡して読み込む

## onLoad

読み込み完了後の処理を書ける

## colorSpace

カラー画像は SRGBColorSpace

# map — 色を表面に貼る

読み込んだテクスチャを material.map に設定すると、表面の色になる

```
const material = new THREE.MeshStandardMaterial({  
  map: tex,           // カラーマップ  
  roughness: 0.6 }); // 表面の粗さ
```

## map

表面の基本色になるカラーマップ

## color

併用時は乗算。通常は白のまま

## needsUpdate

map 差し替え後に true にする

## colorSpace

色がくすむ時にここを確認

# repeat / wrap — 繰り返しと座標

UV座標に沿ってテクスチャを繰り返し、タイル状に敷き詰められる

```
tex.wrapS = THREE.RepeatWrapping;  
tex.wrapT = THREE.RepeatWrapping;  
tex.repeat.set(4, 4); // 4x4 タイル
```

## wrapS / wrapT

繰り返しモードを指定する

## RepeatWrapping

端で画像を繰り返す設定

## repeat.set

縦横の繰り返し回数を指定

## offset

貼り始めの位置をずらす

## PART 02

# 各種マップを使い分ける

PBRマテリアルを支える質感マップ



# 質感 = Color × Normal × Roughness

map

カラー

表面の色や模様を与える

×

normalMap

凹凸

光の反射で凹凸を演出する

×

roughnessMap

粗さ

部分ごとのツヤを変える

これらを重ねると、一枚の画像以上のリアルな質感が生まれる

# 代表的なテクスチャマップ

map

カラーマップ

基本の色・模様

normalMap

法線マップ

凹凸を擬似的に表現

roughnessMap

粗さマップ

部分ごとの光沢差

metalnessMap

金属マップ

金属部分を指定する

aoMap

AOマップ

陰影を加える（要 uv2）

emissiveMap

発光マップ

自ら光る部分を指定

# 色空間とデータの扱い

色画像とデータ画像で colorSpace の扱いを変えるのが重要

## カラーマップ

**SRGBColorSpace**

map ・ emissiveMap など

## データマップ

**Linear** (既定)

normal ・ roughness はそのまま

## 画像形式

**jpg / png / webp**

透過は png ・ webp を使う

```
tex.colorSpace = THREE.SRGBColorSpace; // 色がくすむ時に指定
normalTex.colorSpace = THREE.NoColorSpace; // データは変換しない
```

# 最小コードでテクスチャ

```
import * as THREE from 'three';  
const scene = new THREE.Scene();  
const camera = new THREE.PerspectiveCamera(75, innerWidth/innerHeight, 0.1, 1000);  
const renderer = new THREE.WebGLRenderer({ antialias: true });  
renderer.setSize(innerWidth, innerHeight);  
document.body.appendChild(renderer.domElement);  
  
const tex = new THREE.TextureLoader().load('wood.jpg');  
tex.colorSpace = THREE.SRGBColorSpace;  
const material = new THREE.MeshStandardMaterial({ map: tex });  
const cube = new THREE.Mesh(new THREE.BoxGeometry(1,1,1), material);  
scene.add(cube);  renderer.render(scene, camera);
```

# テクスチャ適用の流れ



読み込みは非同期。完了前は黒く見えることがある →

# 複数マップで木目をリアルに

カラー・法線・粗さを重ねると、一枚絵では出ない立体的な質感に

```
const tl = new THREE.TextureLoader();  
mat.map = tl.load('wood_color.jpg'); // 色  
mat.normalMap = tl.load('wood_normal.jpg'); // 凹凸  
mat.roughnessMap = tl.load('wood_rough.jpg'); // 粗さ
```

## 立体感の深み

凹凸と光沢差で木目が浮かび上がる

## PBRの定番

実写の質感セットはこの3枚が基本

# テクスチャをきれいに出す

ミップマップと異方性フィルタで、遠景や斜めの面のボケ・ジャギを抑える

```
tex.minFilter = THREE.LinearMipmapLinearFilter; // 縮小時  
tex.magFilter = THREE.LinearFilter;           // 拡大時  
tex.anisotropy = renderer.capabilities.getMaxAnisotropy();  
tex.generateMipmaps = true;                   // ミップマップ生成
```

## ぼやける・ジャギが出る時は？

異方性フィルタ（anisotropy）を上げ、ミップマップを有効に。画像サイズは2のべき乗が安全

# よくあるつまずきポイント

## Q 物体が真っ黒になる

読み込み未完了かパス誤り。onLoad で確認する

## Q 色がくすむ・薄い

colorSpace が未設定。SRGBColorSpace を指定

## Q テクスチャがぼやける

anisotropy とミップマップを設定する

## Q タイルが繰り返さない

wrapS / wrapT を RepeatWrapping にする



# 立方体に木目を貼ろう

## Step 1 画像を用意する

木目画像を `public/wood.jpg` に置く

## Step 2 テクスチャを読み込む

`TextureLoader` で load する

## Step 3 `colorSpace` を設定

`SRGBColorSpace` を指定する

## Step 4 `map` に設定する

Standard マテリアルの `map` に渡す

## Step 5 ブラウザで確認

木目の立方体が回転する

**ゴール:** 木目の質感をまとった立方体が、光と影の中で回転する

# まとめ & 次回予告

## 今日のまとめ

- TextureLoader で画像を読み込む
- map に設定して表面に貼る
- カラーは SRGB / データmapは Linear
- repeat・wrap でタイル化
- 複数マップで質感の深みを出す

## 次回: SESSION 05

### カメラ操作とインタラクション

ユーザーの操作に反応するシーンづくりへ。視点を動かし、オブジェクトを選べるようにします。

- OrbitControls で視点を回す・寄る
- Raycaster でクリック選択を判定
- ホバーやタッチ操作への対応